From Flask to Full-Stack: A 3-Month Job-Ready Roadmap

Your instructor's note: I'm not here to write your code for you — I'm here to make sure you never get stuck for more than 30 minutes without knowing why. This course assumes you already know Python basics (variables, functions, loops, lists/dicts). If you don't, spend 3-4 days on a Python refresher before Week 1. Everything else, we build together.

How This Course Works

12 weeks, 4 projects, 1 capstone. Each week has a theme, daily-ish tasks, a mini-project, and a **Debugging Clinic** — real errors you WILL hit, and how to read them.

Time budget: ~15-20 hrs/week (2-3 hrs weekdays, more on weekends). Adjust to your pace — the sequence matters more than the calendar.

Rule #1: Never copy-paste code you don't understand. Type it out. Break it on purpose. Fix it.

Rule #2: Google the error message, not "how do I build X." You're training a debugging instinct, not a memory.

Rule #3: Every project goes on GitHub with a real README, from Week 1 onward. Recruiters look at your commit history before they look at your resume.

MONTH 1 — Backend Foundations with Flask

Week 1: Flask Fundamentals & Routing

Objectives: Understand WSGI, the request-response cycle, routing, and Jinja2 templating.

Concepts to cover:

`Flask()` app factory pattern (use it from day one — don't learn the bad habit of a global app object)

Routes, `@app.route`, dynamic URL segments (`<int:id>`), HTTP methods

`render_template()`, Jinja2 syntax (`{{ }}`, `{% %}`), template inheritance (`base.html`)

Static files (CSS/JS) and Flask's static folder convention

The Flask debugger — turn `debug=True` on and deliberately trigger a 500 error to see the interactive traceback

Hands-on tasks:

Build a "Digital Business Card" site: 3 routes (home, about, projects), one shared base template.

Add a dynamic route `/blog/<int:post_id>` that renders different content based on the ID.

Break it: pass a string to `<int:post_id>` and read the resulting 404. Then explain in your own words why it happened.

Mini-project: Personal portfolio site (static content, no database yet) — deploy locally only for now.

Debugging Clinic — Week 1:

`TemplateNotFound` → 90% of the time it's a folder-naming issue. Check `templates/` is a sibling of your app file, not nested wrong.

`werkzeug.routing.exceptions.BuildError` → you renamed a route function but a `url_for()` call still references the old name.

Blank white page vs stack trace → means `debug=True` isn't set, or you're running via a WSGI server that swallows tracebacks. Fix this first, always.

Resources checklist:

- [ ] Flask official Quickstart docs
- [ ] Jinja2 template docs (just the "for/if/inheritance" sections)
- [ ] MDN: HTTP request methods refresher

Week 2: Forms, Databases & CRUD

Objectives: Persist data. Handle user input safely.

Concepts to cover:

`request.form`, `request.args`, GET vs POST, CSRF concept (why raw forms are risky)
Flask-WTF for forms + validation
SQLAlchemy ORM: models, `db.Column`, relationships (`ForeignKey`, `relationship()`)
Flask-Migrate for schema changes (never manually edit a live schema again)
Full CRUD: Create, Read, Update, Delete on one resource

Hands-on tasks:

Add a `Project` model to your portfolio site (title, description, link, created_at).
Build a form to add new projects, list them on the homepage, and delete them.
Deliberately forget a `db.session.commit()` somewhere and observe that your data "disappears" — this is the #1 beginner Flask+SQLAlchemy bug.

Mini-project: Upgrade the portfolio to store projects in SQLite via SQLAlchemy, with a working add/edit/delete flow.

Debugging Clinic — Week 2:

Data not saving → check `db.session.commit()` is actually called, and that you're not committing inside a conditional that never executes.
`sqlalchemy.exc.OperationalError: no such table` → migration wasn't run, or you're pointing at the wrong `.db` file.
Form submits but nothing changes → check `request.method == 'POST'` is actually being hit; print/log it to confirm.

Resources checklist:

- [ ] Flask-SQLAlchemy quickstart
- [ ] Flask-WTF docs
- [ ] Flask-Migrate docs

Week 3: Authentication, Sessions & Blueprints

Objectives: Let users log in. Structure your app so it doesn't collapse under its own weight.

Concepts to cover:

Password hashing (`werkzeug.security` — never store plaintext, ever)
Flask-Login: `UserMixin`, `login_user`, `login_required`, sessions vs cookies
Blueprints — splitting `auth`, `main`, `api` into separate modules
Flash messages for user feedback

Hands-on tasks:

Add a `User` model, registration and login pages.
Restrict "add project" to logged-in users only.
Refactor your single-file app into blueprints (`/auth`, `/main`) — this is the moment your app starts looking like a real codebase.

Mini-project: Multi-user portfolio platform — each user has their own login and their own set of projects.
Debugging Clinic — Week 3:
Logged-in user gets logged out randomly → usually `SECRET_KEY` isn't set consistently (or changes on every restart).
`RuntimeError: working outside of application context` → you're using `current_app` or `db` outside a request; needs `app.app_context()`.
Circular import errors after adding blueprints → this is THE classic Flask-at-scale bug. Learn the app-factory + `__init__.py` pattern properly here, not later.
Resources checklist:
- [ ] Flask-Login docs
- [ ] Flask Blueprints official tutorial ("Flaskr" tutorial is gold — do the whole thing)

Week 4: REST APIs, Testing & Project 1
Objectives: Build an API a frontend (or another dev) could actually consume. Start testing.
Concepts to cover:
JSON responses (`jsonify`), status codes, REST conventions
API structure: `/api/v1/projects` GET/POST/PUT/DELETE
`pytest` basics + Flask's test client
Postman or `curl` for manual API testing
Hands-on tasks:
Convert your portfolio app's project CRUD into a JSON API alongside the HTML views.
Write 5 pytest tests: one per CRUD action, plus one for "unauthorized access is rejected."
Test your API with Postman before you ever build a frontend for it — decouple your thinking now.
PROJECT 1 (end of Month 1): A multi-user portfolio/blog platform with auth, full CRUD, a REST API layer, and a test suite. Push to GitHub with a proper README (setup instructions, screenshots, tech stack).
Debugging Clinic — Week 4:
API returns HTML error page instead of JSON → you need an `errorhandler` that returns JSON for `/api/*` routes specifically.
Tests pass locally, fail in CI later → usually a hardcoded file path or missing test database config. Get in the habit of using a separate test config now.

MONTH 2 — Full-Stack Integration
Week 5: Frontend Fundamentals for Backend Devs
Objectives: Get comfortable enough with HTML/CSS/JS that Flask templates stop feeling like a black box.
Concepts to cover:
Semantic HTML, CSS box model, Flexbox/Grid basics
A CSS framework (Tailwind or Bootstrap — pick one, don't dabble in both)
Jinja2 macros and advanced template inheritance (reusable components)
Hands-on tasks:
Rebuild your portfolio's UI using Tailwind (or Bootstrap) instead of raw CSS.
Create a Jinja2 macro for a "project card" component, reused across pages.
Debugging Clinic — Week 5:

Styles "not applying" → check CDN link is actually loading (Network tab in DevTools), and CSS specificity conflicts.
Layout breaks on mobile → you skipped a viewport meta tag or didn't test responsive breakpoints.

Week 6: JavaScript, Fetch & Dynamic Frontends

Objectives: Make pages interactive without full reloads.
Concepts to cover:
Vanilla JS DOM manipulation, event listeners
`fetch()` for calling your own Flask API (async/await)
Updating the DOM based on JSON responses
CORS — what it is and why it exists (you'll hit this the moment frontend and backend run on different ports)
Hands-on tasks:
Add a "delete project" button that calls your API via `fetch()` and removes the card from the page without a reload.
Add client-side form validation before submission.
Deliberately trigger a CORS error by calling your API from a different origin, then fix it with `flask-cors`.
Debugging Clinic — Week 6:
`fetch` silently fails → always `.catch()` and log errors; check the Network tab for the actual status code.
CORS error in console → the browser is protecting the user, not you being blocked arbitrarily; fix it server-side with proper headers, never by disabling browser security.

Week 7: Databases, Deeper — PostgreSQL & Relationships

Objectives: Move off SQLite. Model real relationships.
Concepts to cover:
PostgreSQL setup (local or via Docker container)
One-to-many and many-to-many relationships in SQLAlchemy
Query optimization basics: `N+1 query problem`, `joinedload`
Environment variables for DB credentials (`python-dotenv`)
Hands-on tasks:
Migrate your app's database from SQLite to PostgreSQL.
Add a many-to-many relationship (e.g., Projects ↔ Tags).
Deliberately write an N+1 query, watch it in Flask-SQLAlchemy's query logging, then fix it with eager loading.
Debugging Clinic — Week 7:
`sqlalchemy.exc.IntegrityError` → almost always a constraint violation (unique, foreign key, not-null). Read the actual constraint name in the error.
App works locally, breaks after "deploy prep" → 9 times out of 10 it's hardcoded credentials vs environment variables. Get disciplined about `.env` + `.gitignore` now.

Week 8: File Uploads, Background Jobs & Project 2

Objectives: Handle real-world app needs — uploads, async tasks, notifications.
Concepts to cover:
File upload handling, validation, storage (local first, then intro to S3-style cloud storage conceptually)

Sending email (Flask-Mail or an API like SendGrid)
Background/async tasks with Celery + Redis (conceptual depth is enough here — don't over-invest)
Hands-on tasks:
Add profile picture upload to user accounts, with file-type/size validation.
Send a confirmation email on registration.
Move one slow task (e.g., "send email") to a background job so it doesn't block the request.
PROJECT 2 (end of Month 2): A fullstack task manager or mini-blog with: PostgreSQL, many-to-many relationships, file uploads, a JS-driven dynamic frontend calling your API, and background email notifications. This is your first genuinely "portfolio-worthy" app.
Debugging Clinic — Week 8:
Uploaded file "disappears" after deploy → most PaaS platforms have ephemeral filesystems; this is why cloud storage exists. Understand this now, even before you deploy.
Celery worker not picking up tasks → check Redis is actually running and the worker was restarted after code changes.

MONTH 3 — Production, Testing & Job Readiness

Week 9: Testing, Debugging & Logging Like a Professional

Objectives: Build the instincts that separate "it works on my machine" devs from hireable ones.
Concepts to cover:
pytest fixtures, `conftest.py`, test coverage
Structured logging (`logging` module, not `print()`)
Using `pdb`/`breakpoint()` for step-through debugging
Reading stack traces bottom-up (the actual error is at the bottom, not the top)
Hands-on tasks:
Get your Project 2 test coverage to a meaningful level (aim for core business logic, not 100% vanity coverage).
Replace every `print()` debug statement in your codebase with proper logging.
Intentionally introduce 3 bugs into a teammate's (or your own old) code and fix them using only `breakpoint()` — no guessing.
Debugging Clinic — Week 9 (this is the whole week, really):
The #1 skill: when you hit an error, read the LAST line of the traceback first, then walk upward to find where in *your* code (not library code) it originated.
Keep a personal "bug log" — one line per bug: what broke, why, how you found it. By week 12 this becomes your interview war-stories bank.

Week 10: Deployment & DevOps Basics

Objectives: Ship it. A project that only runs on `localhost` doesn't exist to an employer.
Concepts to cover:
Docker: writing a `Dockerfile` for a Flask app, `docker-compose` for app + DB
Gunicorn as a production WSGI server (never `app.run()` in production)
Deploying to Render, Railway, or Fly.io (pick one — all have free tiers)
Environment variables and secrets in production
Basic CI with GitHub Actions (run tests automatically on push)

Hands-on tasks:
Dockerize Project 2 fully (app + Postgres + Redis via docker-compose).
Deploy it live, with a real URL you can put on your resume.
Set up a GitHub Actions workflow that runs your pytest suite on every push.
Debugging Clinic — Week 10:
"Works in Docker locally, crashes in production" → check env vars are actually set on the host platform, not just in your local `.env`.
502/504 errors after deploy → usually Gunicorn worker count/timeout misconfigured, or the app is binding to the wrong port (`0.0.0.0` vs `127.0.0.1`).

Week 11: Security, Performance & Caching

Objectives: Think like someone who'll be trusted with production data.
Concepts to cover:
OWASP basics: SQL injection (why ORM helps, but doesn't save you from raw queries), XSS, CSRF protection
Rate limiting (Flask-Limiter)
Caching with Redis (cache expensive queries)
HTTPS, secure cookies, security headers (Flask-Talisman)
Hands-on tasks:
Audit Project 2 for at least 3 real security issues and fix them.
Add rate limiting to your login endpoint (prevents brute-force).
Cache one expensive query/endpoint and measure the performance difference.
Debugging Clinic — Week 11:
CSRF errors on legitimate form submits → token expired or not included in AJAX headers; this is a common fetch+Flask-WTF gotcha.
Cache showing stale data → you forgot cache invalidation on write; "cache invalidation is one of the two hard problems in computer science" for a reason.

Week 12: Capstone Project + Job Readiness

Objectives: Build something interview-worthy. Package yourself.
CAPSTONE PROJECT: Design and build a fullstack application from scratch, combining everything: auth, relational DB, REST API, dynamic JS frontend, file handling, background jobs, tests, Docker, and live deployment. Suggested ideas (or bring your own): a job-application tracker, a recipe-sharing platform, a small SaaS tool for a niche you know well. **Pick something you'd actually use — it shows in interviews.**
Job-readiness tasks:
Write a strong README for every project (problem, tech stack, screenshots, live link, what you'd improve).
Prepare 3-5 "bug stories" from your debugging log — interviewers love hearing how you solved real problems, not just what you built.
Do a full code review pass on your capstone: consistent naming, no dead code, no secrets committed, sensible folder structure.
Practice explaining your architecture out loud in 2 minutes — this is basically every technical interview's opening question.
Update your resume/LinkedIn/GitHub profile README with the three projects, live links, and tech stack.

The Standing Debugging Playbook (keep this open in a tab)

Read the actual error, bottom of the traceback first. Don't skim — the exception type and message usually tell you exactly what's wrong.
Reproduce it minimally. Can you trigger the bug with the smallest possible input? If not, you don't understand it yet.
Isolate: is it Python, Flask, the DB, or the browser? Check each layer separately — `print`/log values right before the failure point.
Check the boring stuff first: Did you save the file? Restart the server? Activate the virtualenv? Is the `.env` actually loaded? (This solves more bugs than people admit.)
Use `breakpoint()` to pause execution and inspect real variable state instead of guessing.
Diff against a working version (git stash, git diff) if something that worked yesterday broke today.
Search the exact error text, not a vague description — in quotes, with the library name.
Explain the bug out loud to someone (or a rubber duck). This surfaces the wrong assumption 80% of the time.

Common Flask Error Cheat-Sheet

Error	Usual Cause
`TemplateNotFound`	Wrong folder structure or typo in filename
`werkzeug.routing.exceptions.BuildError`	`url_for()` referencing a renamed/missing endpoint
`RuntimeError: working outside of application context`	DB/app object used outside a request or `app_context()`
`sqlalchemy.exc.OperationalError`	Wrong DB path, migrations not run, DB not running
`sqlalchemy.exc.IntegrityError`	Constraint violation (unique/foreign key/not-null)
CORS error in browser console	Missing/misconfigured CORS headers, not a "block" on you
502/504 after deploy	Gunicorn misconfig, wrong bind address, worker timeout
CSRF token errors	Expired token or missing header in AJAX requests
"Works locally, breaks in prod"	Environment variables/secrets not set on the host

Suggested Weekly Rhythm

Mon-Fri: 2-3 focused hours — 1 hr concept/reading, 1-2 hrs hands-on coding
Sat: Mini-project work + review your bug log from the week
Sun: Rest, or light reading only — burnout kills more coding journeys than difficulty does

Bring me your error messages, your stuck points, and your half-finished projects any time — that's what office hours are for. Let's start with Week 1.